

```
from google.colab import files
uploaded = files.upload()
```

 No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 1\ iris.csv to 1\ iris.csv

```
import pandas as pd
df= pd.read_csv("1\ iris.csv")
df
```



	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
df.isnull().sum()
```



	0
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

```
import numpy as np
from scipy import stats
z_scores = np.abs(stats.zscore(df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]))
outliers_detected = (z_scores > 3).sum()
print(outliers_detected)
```

 1

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

```
df = pd.read_csv("1\ iris.csv")
```

```
iris_features = df.drop(columns=['species'])
```

```
standard_scaler = StandardScaler()
iris_standardized = standard_scaler.fit_transform(iris_features)
```

```
iris_standardized_df = pd.DataFrame(iris_standardized, columns=iris_features.columns)
```

```
print(iris_standardized_df.head())
```



	sepal_length	sepal_width	petal_length	petal_width
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

```
from sklearn.preprocessing import MinMaxScaler
minmax_scaler = MinMaxScaler()
```

```
iris_normalized = minmax_scaler.fit_transform(df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']])
```

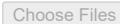
```
df.head(3)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa

```
import pandas as pd
pd.DataFrame({
  "Text": [
    "The petal is wide and short.",
    "Narrow petals with medium sepal.",
    "Species may be setosa or versicolor."
  ]
})
```

	Text
0	The petal is wide and short.
1	Narrow petals with medium sepal.
2	Species may be setosa or versicolor.

```
from google.colab import files
uploaded = files.upload()
```

 No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
df = pd.read_csv("2) Stock Prices Data Set.csv")
df
```

	symbol	date	open	high	low	close	volume
0	AAL	2014-01-02	25.0700	25.8200	25.0600	25.3600	8998943
1	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957
2	AAP	2014-01-02	110.3600	111.8800	109.2900	109.7400	542711
3	ABBV	2014-01-02	52.1200	52.3300	51.5200	51.9800	4569061
4	ABC	2014-01-02	70.1100	70.2300	69.4800	69.8900	1148391
...	...	...	...	...	...	...	...
497467	XYL	2017-12-29	68.5300	68.8000	67.9200	68.2000	1046677
497468	YUM	2017-12-29	82.6400	82.7100	81.5900	81.6100	1347613
497469	ZBH	2017-12-29	121.7500	121.9500	120.6200	120.6700	1023624
497470	ZION	2017-12-29	51.2800	51.5500	50.8100	50.8300	1261916
497471	ZTS	2017-12-29	72.5500	72.7600	72.0400	72.0400	1704122

497472 rows × 7 columns

```
df.isnull().sum()
```

	0
symbol	0
date	0
open	11
high	8
low	8
close	0
volume	0
dtype:	int64

```
df['open'] = df['open'].fillna(df['open'].mean())

from scipy import stats
import numpy as np # Import numpy as it is used in this cell

# Define the numeric columns
numeric_cols = ['open', 'high', 'low', 'close', 'volume']

z_scores = np.abs(stats.zscore(df[numeric_cols]))
outliers_detected = (z_scores > 3).sum()

print(outliers_detected) # Add print statement to display the output
```

↗ 21934

```
numeric_cols = ['open', 'high', 'low', 'close', 'volume']

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

z_scores = np.abs(stats.zscore(df[numeric_cols]))

outliers_detected = (z_scores > 3).sum(axis=0)

outliers_dict = dict(zip(numeric_cols, outliers_detected))
print("Outliers:\n")
print("Column\tOutliers")
for col, count in outliers_dict.items():
    print(f"{col}\t{count}")
```

↗ Outliers:

Column	Outliers
open	6986
high	7003
low	6962
close	6978
volume	7970

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

scaler = StandardScaler()
standardized = scaler.fit_transform(df[numeric_cols])

standardized_df = pd.DataFrame(standardized, columns=numeric_cols)

print(standardized_df.head())
```

↗

	open	high	low	close	volume
0	-0.603945	-0.599276	-0.601496	-0.601239	0.576440
1	-0.068685	-0.073863	-0.066544	-0.072439	6.625058
2	0.236599	0.241884	0.236030	0.230318	-0.450782
3	-0.337363	-0.340164	-0.338396	-0.338901	0.038319
4	-0.160070	-0.165207	-0.159814	-0.162400	-0.377207

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

scaler = MinMaxScaler()
normalized = scaler.fit_transform(df[numeric_cols])

normalized_df = pd.DataFrame(normalized, columns=numeric_cols)

print(normalized_df.head())
```

↗

	open	high	low	close	volume
0	0.011482	0.011678	0.011585	0.011610	0.014556
1	0.038075	0.037693	0.038041	0.037818	0.095096
2	0.053242	0.053327	0.053004	0.052823	0.000878
3	0.024726	0.024508	0.024597	0.024612	0.007390
4	0.033534	0.033170	0.033428	0.033359	0.001858

```
df.head(3) # Structured Data
```

	symbol	date	open	high	low	close	volume
0	AAL	2014-01-02	25.0700	25.8200	25.0600	25.3600	8998943
1	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957
2	AAP	2014-01-02	110.3600	111.8800	109.2900	109.7400	542711

```
import pandas as pd
pd.DataFrame(
    {"Text": ["AAPL stock closed higher today.",
              "Volume spike expected on TSLA earnings.",
              "Markets open lower amid inflation fears."]}
)
```

	Text
0	AAPL stock closed higher today.
1	Volume spike expected on TSLA earnings.
2	Markets open lower amid inflation fears.

```
from google.colab import files
uploaded = files.upload()
```

Choose Files

 No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
df= pd.read_csv("3) Sentiment dataset.csv")
df
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweet
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5
2	2	2	Just finished an amazing workout! 🥰 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20
3	3	3	Excited about the upcoming weekend getaway! ...	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Travel #Adventure	8
4	4	4	Trying out a new recipe for dinner tonight. ...	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cooking #Food	12
...	...	...	...	...	...	...	...	...	...
727	728	732	Collaborating on a science project that receiv...	Happy	2017-08-18 18:20:00	ScienceProjectSuccessHighSchool	Facebook	#ScienceFairWinner #HighSchoolScience	20
728	729	733	Attending a surprise birthday party organized ...	Happy	2018-06-22 14:15:00	BirthdayPartyJoyHighSchool	Instagram	#SurpriseCelebration #HighSchoolFriendship	25
729	730	734	Successfully fundraising for a school charity ...	Happy	2019-04-05 17:30:00	CharityFundraisingTriumphHighSchool	Twitter	#CommunityGiving #HighSchoolPhilanthropy	22

```
df.isnull().sum()
```

```

0
Unnamed: 0.1  0
Unnamed: 0    0
Text          0
Sentiment     0
Timestamp     0
User          0
Platform      0
Hashtags      0
Retweets      0
Likes         0
Country       0
Year          0
Month         0
Day           0
Hour          0

```

```
dtype: int64
```

```

from scipy import stats
z_scores = np.abs(stats.zscore(df[['Likes', 'Retweets'])))
outliers_detected = (z_scores > 3).sum()

```

```

import pandas as pd
from sklearn.preprocessing import StandardScaler

```

```
df = pd.read_csv("3) Sentiment dataset.csv")
```

```
features = df[['Likes', 'Retweets']]
```

```

scaler = StandardScaler()
standardized = scaler.fit_transform(features)

```

```
standardized_df = pd.DataFrame(standardized, columns=['Likes', 'Retweets'])
```

```
print(standardized_df.head())
```

```

Likes  Retweets
0 -0.916295 -0.922303
1 -2.336727 -2.339444
2 -0.206079 -0.213733
3 -1.981619 -1.914302
4 -1.271403 -1.347445

```

```

import pandas as pd
from sklearn.preprocessing import StandardScaler

```

```
df = pd.read_csv("3) Sentiment dataset.csv")
```

```

scaler = StandardScaler()
standardized = scaler.fit_transform(df[['Likes', 'Retweets']])

```

```
standardized_df = pd.DataFrame(standardized, columns=['Likes', 'Retweets'])
```

```
standardized_df_rounded = standardized_df.round(2)
```

```

print("Sample Output:")
print(standardized_df_rounded.head())

```

```

Sample Output:
Likes  Retweets
0 -0.92    -0.92
1 -2.34    -2.34
2 -0.21    -0.21
3 -1.98    -1.91
4 -1.27    -1.35

```

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("3) Sentiment dataset.csv")

features = df[['Likes', 'Retweets']]

scaler = MinMaxScaler()
normalized = scaler.fit_transform(features)

normalized_df = pd.DataFrame(normalized, columns=['Likes', 'Retweets'])

normalized_df_rounded = normalized_df.round(2)

print("Sample Output:")
print(normalized_df_rounded.head())
```

↗ Sample Output:

	Likes	Retweets
0	0.29	0.29
1	0.00	0.00
2	0.43	0.43
3	0.07	0.09
4	0.21	0.20

```
df[['Likes', 'Retweets', 'Country', 'Year']]
```

↗

	Likes	Retweets	Country	Year
0	30.0	15.0	USA	2023
1	10.0	5.0	Canada	2023
2	40.0	20.0	USA	2023
3	15.0	8.0	UK	2023
4	25.0	12.0	Australia	2023
...	...	...	...	...
727	39.0	20.0	UK	2017
728	48.0	25.0	USA	2018
729	42.0	22.0	Canada	2019
730	43.0	21.0	UK	2020
731	47.0	24.0	USA	2020

732 rows × 4 columns

```
df[['Text', 'Hashtags']]
```

↗

	Text	Hashtags
0	Enjoying a beautiful day at the park! ...	#Nature #Park
1	Traffic was terrible this morning. ...	#Traffic #Morning
2	Just finished an amazing workout! 🏋️ ...	#Fitness #Workout
3	Excited about the upcoming weekend getaway! ...	#Travel #Adventure
4	Trying out a new recipe for dinner tonight. ...	#Cooking #Food
...	...	...
727	Collaborating on a science project that receiv...	#ScienceFairWinner #HighSchoolScience
728	Attending a surprise birthday party organized ...	#SurpriseCelebration #HighSchoolFriendship
729	Successfully fundraising for a school charity ...	#CommunityGiving #HighSchoolPhilanthropy
730	Participating in a multicultural festival, cel...	#CulturalCelebration #HighSchoolUnity
731	Organizing a virtual talent show during challe...	#VirtualEntertainment #HighSchoolPositivity

732 rows × 2 columns

```
from google.colab import files
uploaded = files.upload()
```

↗ No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
df= pd.read_csv("4) house Prediction Data Set.csv")
df
```

 **0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00**

0	0.02731 0.00 7.070 0 0.4690 6.4210 78...
1	0.02729 0.00 7.070 0 0.4690 7.1850 61...
2	0.03237 0.00 2.180 0 0.4580 6.9980 45...
3	0.06905 0.00 2.180 0 0.4580 7.1470 54...
4	0.02985 0.00 2.180 0 0.4580 6.4300 58...
...	...
500	0.06263 0.00 11.930 0 0.5730 6.5930 69...
501	0.04527 0.00 11.930 0 0.5730 6.1200 76...
502	0.06076 0.00 11.930 0 0.5730 6.9760 91...
503	0.10959 0.00 11.930 0 0.5730 6.7940 89...
504	0.04741 0.00 11.930 0 0.5730 6.0300 80...

505 rows × 1 columns

```
df.isnull().sum()
```

 **0**

0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00	0
---	----------

dtype: int64

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from scipy import stats

raw_df = pd.read_csv("4) house Prediction Data Set.csv", header=None)
```

```
if raw_df.shape[1] == 1:
    df = raw_df[0].str.split(",", expand=True)
else:
    df = raw_df.copy()
```

```
df = df.apply(pd.to_numeric, errors='coerce')
```

```
missing = df.isnull().sum()
print("Missing Values:\n", missing)
```

 **Missing Values:**

0	506
----------	------------

dtype: int64

```
df_filled = df.fillna(df.mean(numeric_only=True))
```

```
numeric_cols = df_filled.select_dtypes(include=np.number).columns
z_scores = np.abs(stats.zscore(df_filled[numeric_cols]))
outliers_detected = (z_scores > 3).sum(axis=0) # Fix: Calculate outliers for each column
```

```
outliers_dict = dict(zip(numeric_cols, outliers_detected)) # Fix: Create a dictionary from the results
print("\nOutliers Detected:\n", outliers_dict) # Fix: Print the dictionary
```

 **Outliers Detected:**

{0: np.int64(0)}

```
scaler_std = StandardScaler()
standardized = scaler_std.fit_transform(df_filled[numeric_cols])
standardized_df = pd.DataFrame(standardized, columns=numeric_cols)
print("\nStandardized Data (First 5 Rows):\n", standardized_df.head())
```

 **Standardized Data (First 5 Rows):**

0
0 NaN

```
1 NaN
2 NaN
3 NaN
4 NaN
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1101: RuntimeWarning: invalid value encountered in divide
  updated_mean = (last_sum + new_sum) / updated_sample_count
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1106: RuntimeWarning: invalid value encountered in divide
  T = new_sum / new_sample_count
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1126: RuntimeWarning: invalid value encountered in divide
  new_unnormalized_variance -= correction**2 / new_sample_count
```

```
scaler_minmax = MinMaxScaler()
normalized = scaler_minmax.fit_transform(df_filled[numeric_cols])
normalized_df = pd.DataFrame(normalized, columns=numeric_cols)
print("\nNormalized Data (First 5 Rows):\n", normalized_df.head())
```



Normalized Data (First 5 Rows):

```
0
0 NaN
1 NaN
2 NaN
3 NaN
4 NaN
/usr/local/lib/python3.11/dist-packages/sklearn/utils/_array_api.py:776: RuntimeWarning: All-NaN slice encountered
  return xp.asarray(numpy.nanmin(X, axis=axis))
/usr/local/lib/python3.11/dist-packages/sklearn/utils/_array_api.py:793: RuntimeWarning: All-NaN slice encountered
  return xp.asarray(numpy.nanmax(X, axis=axis))
```

```
print("\nStructured Sample:\n", df_filled.head(3))
print("\nUnstructured Example:\n")
print(pd.DataFrame({
    "Text": [
        "3-bedroom apartment with sea view.",
        "2BHK flat near metro station with balcony.",
        "Spacious house with private pool and gym."
    ]
}))
```



Structured Sample:

```
0
0 NaN
1 NaN
2 NaN
```

Unstructured Example:

	Text
0	3-bedroom apartment with sea view.
1	2BHK flat near metro station with balcony.
2	Spacious house with private pool and gym.

```
import pandas as pd
```

```
df = pd.read_csv("1) iris.csv")
```

```
iris_numeric = df.drop(columns=['species'])
```

```
mean_vals = iris_numeric.mean()
median_vals = iris_numeric.median()
mode_vals = iris_numeric.mode().iloc[0]
std_vals = iris_numeric.std()
```

```
print("Mean:")
print(mean_vals.round(2))
```

```
print("Median:")
print(median_vals.round(2))
```

```
print("Mode:")
print(mode_vals.round(2))
```

```
print("Standard Deviation:")
print(std_vals.round(2))
```



```
Mean:
sepal_length    5.84
sepal_width     3.05
petal_length    3.76
petal_width     1.20
dtype: float64
Median:
sepal_length    5.80
```

```

sepal_width    3.00
petal_length   4.35
petal_width    1.30
dtype: float64
Mode:
sepal_length   5.0
sepal_width    3.0
petal_length   1.5
petal_width    0.2
Name: 0, dtype: float64
Standard Deviation:
sepal_length   0.83
sepal_width    0.43
petal_length   1.76
petal_width    0.76
dtype: float64

```

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

```

```
df = pd.read_csv("1) iris.csv")
```

```
correlation_matrix = df.corr(numeric_only=True)
```

```

print("Correlation Matrix:")
print(correlation_matrix.round(2))

```

```

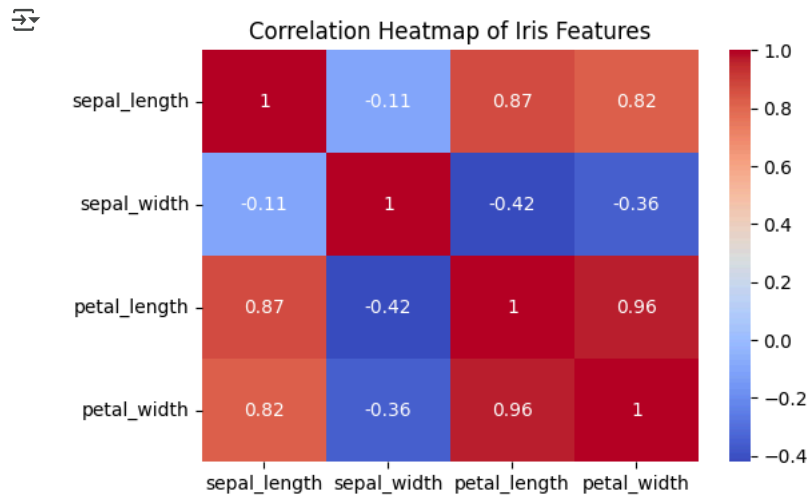
↗↘ Correlation Matrix:
      sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.00      -0.11         0.87         0.82
sepal_width      -0.11       1.00        -0.42        -0.36
petal_length       0.87      -0.42         1.00         0.96
petal_width       0.82      -0.36         0.96         1.00

```

```

plt.figure(figsize=(6, 4))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap of Iris Features")
plt.show()

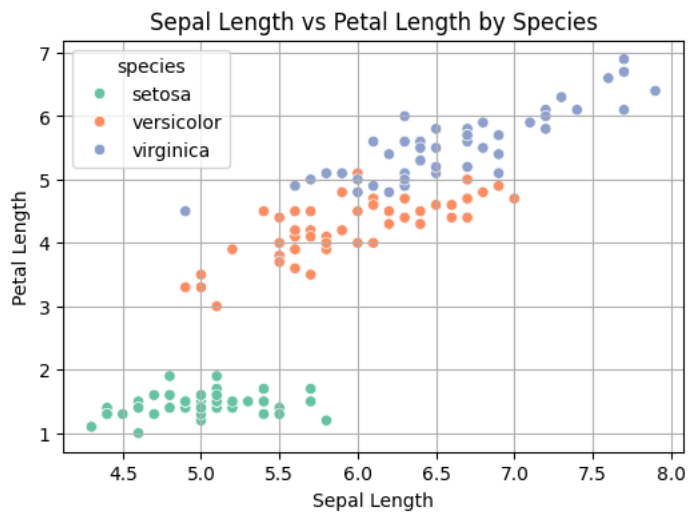
```



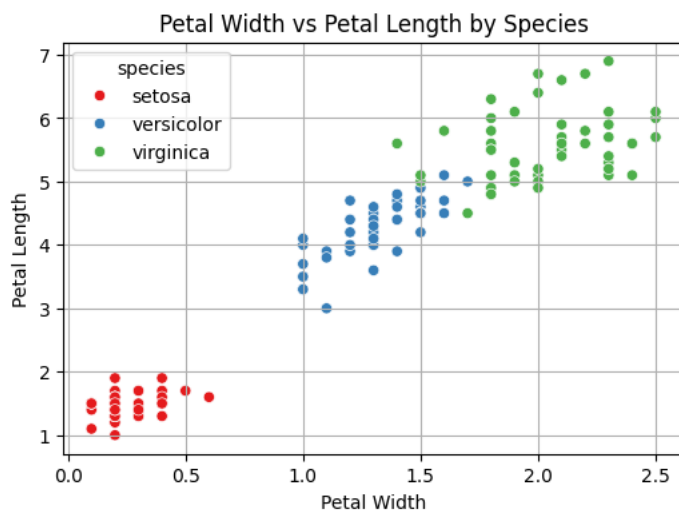
```

plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='sepal_length', y='petal_length', hue='species', palette='Set2')
plt.title("Sepal Length vs Petal Length by Species")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.grid(True)
plt.show()

```



```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='petal_width', y='petal_length', hue='species', palette='Set1')
plt.title("Petal Width vs Petal Length by Species")
plt.xlabel("Petal Width")
plt.ylabel("Petal Length")
plt.grid(True)
plt.show()
```



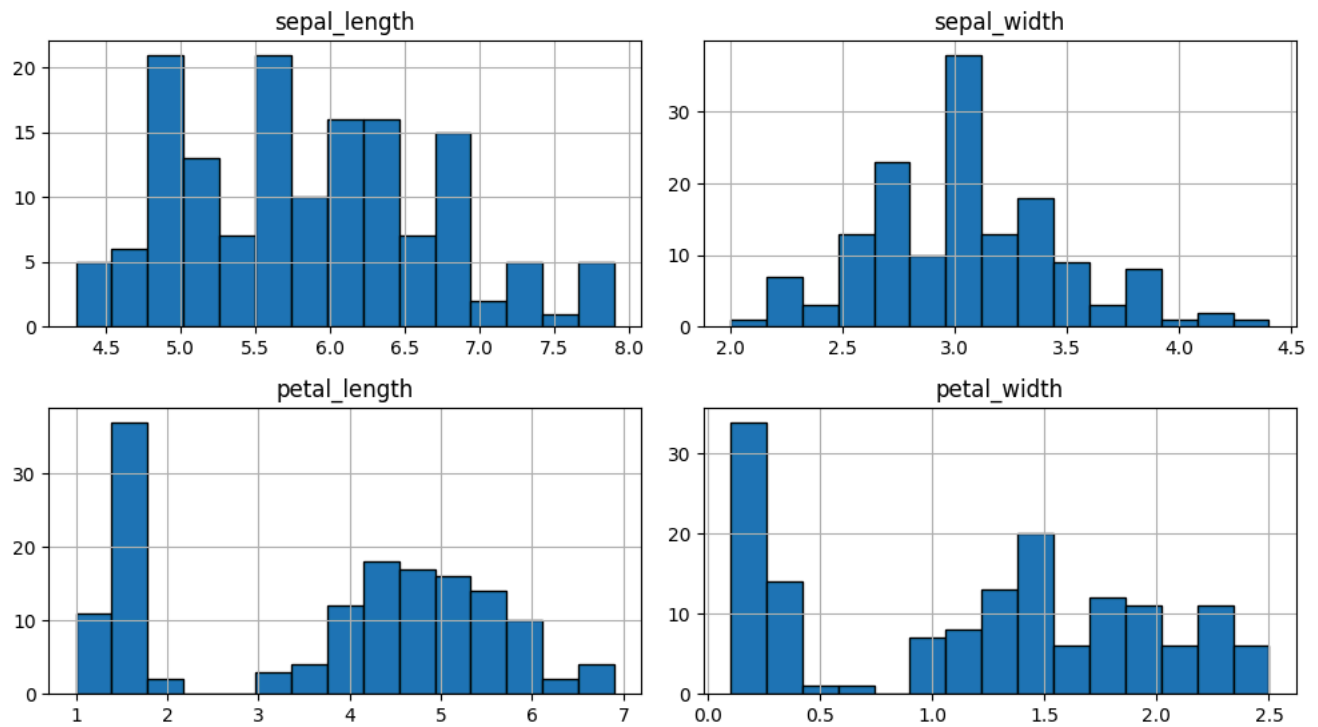
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("1) iris.csv") # Replace with your correct path

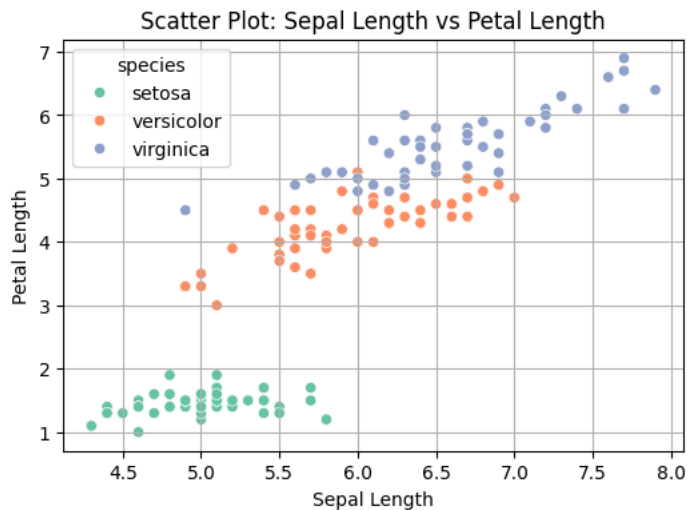
df.drop(columns=['species']).hist(edgecolor='black', figsize=(10, 6), bins=15)
plt.suptitle("Histogram of All Numeric Features", fontsize=16)
plt.tight_layout()
plt.show()
```



Histogram of All Numeric Features

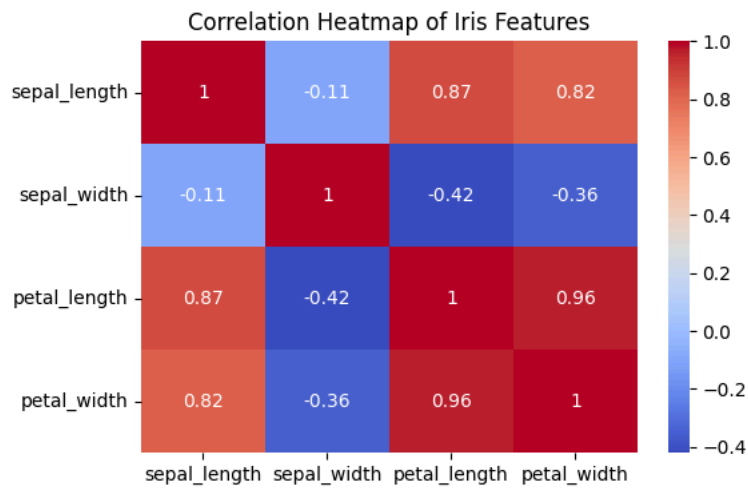


```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='sepal_length', y='petal_length', hue='species', palette='Set2')
plt.title("Scatter Plot: Sepal Length vs Petal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.grid(True)
plt.show()
```



```
correlation = df.corr(numeric_only=True)
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(correlation, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap of Iris Features")
plt.show()
```



```
import pandas as pd

df = pd.read_csv("1) iris.csv")

correlation_matrix = df.corr(numeric_only=True)

print("Correlation Matrix:\n")
print(correlation_matrix.round(3))
```



Correlation Matrix:

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000	-0.109	0.872	0.818
sepal_width	-0.109	1.000	-0.421	-0.357
petal_length	0.872	-0.421	1.000	0.963
petal_width	0.818	-0.357	0.963	1.000

```
import pandas as pd
from scipy import stats
```

```
df = pd.read_csv("1) iris.csv")
```

```
x = df['sepal_length']
y = df['petal_length']
```

```
slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)
```

```
print("Linear Regression: Sepal Length vs Petal Length\n")
print(f"Regression Equation: y = {slope:.2f} * sepal_length + ({intercept:.2f})")
print(f"Correlation Coefficient (r): {r_value:.3f}")
print(f"P-value: {p_value:.1e}")
```



Linear Regression: Sepal Length vs Petal Length

```
Regression Equation: y = 1.86 * sepal_length + (-7.10)
Correlation Coefficient (r): 0.872
P-value: 1.0e-47
```

```
from google.colab import files
uploaded = files.upload()
```



No file chosen
enable.

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to

```
import pandas as pd
df = pd.read_csv("2) Stock Prices Data Set.csv")
df
```



	symbol	date	open	high	low	close	volume
0	AAL	2014-01-02	25.0700	25.8200	25.0600	25.3600	8998943
1	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957
2	AAP	2014-01-02	110.3600	111.8800	109.2900	109.7400	542711
3	ABBV	2014-01-02	52.1200	52.3300	51.5200	51.9800	4569061
4	ABC	2014-01-02	70.1100	70.2300	69.4800	69.8900	1148391
...	...	...	...	...	...	...	...
497467	XYL	2017-12-29	68.5300	68.8000	67.9200	68.2000	1046677
497468	YUM	2017-12-29	82.6400	82.7100	81.5900	81.6100	1347613
497469	ZBH	2017-12-29	121.7500	121.9500	120.6200	120.6700	1023624
497470	ZION	2017-12-29	51.2800	51.5500	50.8100	50.8300	1261916
497471	ZTS	2017-12-29	72.5500	72.7600	72.0400	72.0400	1704122

497472 rows × 7 columns

```
import pandas as pd

df = pd.read_csv("2) Stock Prices Data Set.csv")

numeric_cols = df.select_dtypes(include='number')

mean_vals = numeric_cols.mean().round(2)
median_vals = numeric_cols.median().round(2)
mode_vals = numeric_cols.mode().iloc[0].round(2)
std_vals = numeric_cols.std().round(2)

print("Mean:")
print(mean_vals)

print("Median:")
print(median_vals)

print("Mode:")
print(mode_vals)

print("Standard Deviation:")
print(std_vals)
```



```
Mean:
open      86.35
high      87.13
low       85.55
close     86.37
volume    4253610.90
dtype: float64
Median:
open      64.97
high      65.56
low       64.35
close     64.98
volume    2084896.50
dtype: float64
Mode:
open      70.0
high      72.0
low       77.0
close     34.5
volume    1241019.0
Name: 0, dtype: float64
Standard Deviation:
open      101.47
high      102.31
low       100.57
close     101.47
volume    8232139.24
dtype: float64
```

```
import pandas as pd

df = pd.read_csv("2) Stock Prices Data Set.csv")

numeric_cols = df.select_dtypes(include='number')

correlation_matrix = numeric_cols.corr()
```

```
print("Correlation Matrix:")
print(correlation_matrix.round(3))
```

```
↗ Correlation Matrix:
      open  high  low  close  volume
open  1.000  1.000  1.000  1.000  -0.149
high  1.000  1.000  1.000  1.000  -0.148
low   1.000  1.000  1.000  1.000  -0.149
close 1.000  1.000  1.000  1.000  -0.149
volume -0.149 -0.148 -0.149 -0.149  1.000
```

```
import pandas as pd
from scipy import stats
```

```
df = pd.read_csv("2) Stock Prices Data Set.csv")
```

```
print("Missing Values:")
print(df[['open', 'close']].isnull().sum())
```

```
df_clean = df[['open', 'close']].dropna()
```

```
if df_clean['open'].nunique() == 1 or df_clean['close'].nunique() == 1:
    print("One of the columns has a constant value. Regression is not possible")
else:
    slope, intercept, r_value, p_value, std_err = stats.linregress(df_clean['open'], df_clean['close'])
    print("Linear Regression: open vs close")
    print(f"Regression Equation: y = {slope:.2f} * open + ({intercept:.2f})")
    print(f"Correlation Coefficient (r): {r_value:.3f}")
    print(f"P-value: {p_value:.1e}")
```

```
↗ Missing Values:
open      11
close       0
dtype: int64
Linear Regression: open vs close
Regression Equation: y = 1.00 * open + (0.03)
Correlation Coefficient (r): 1.000
P-value: 0.0e+00
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

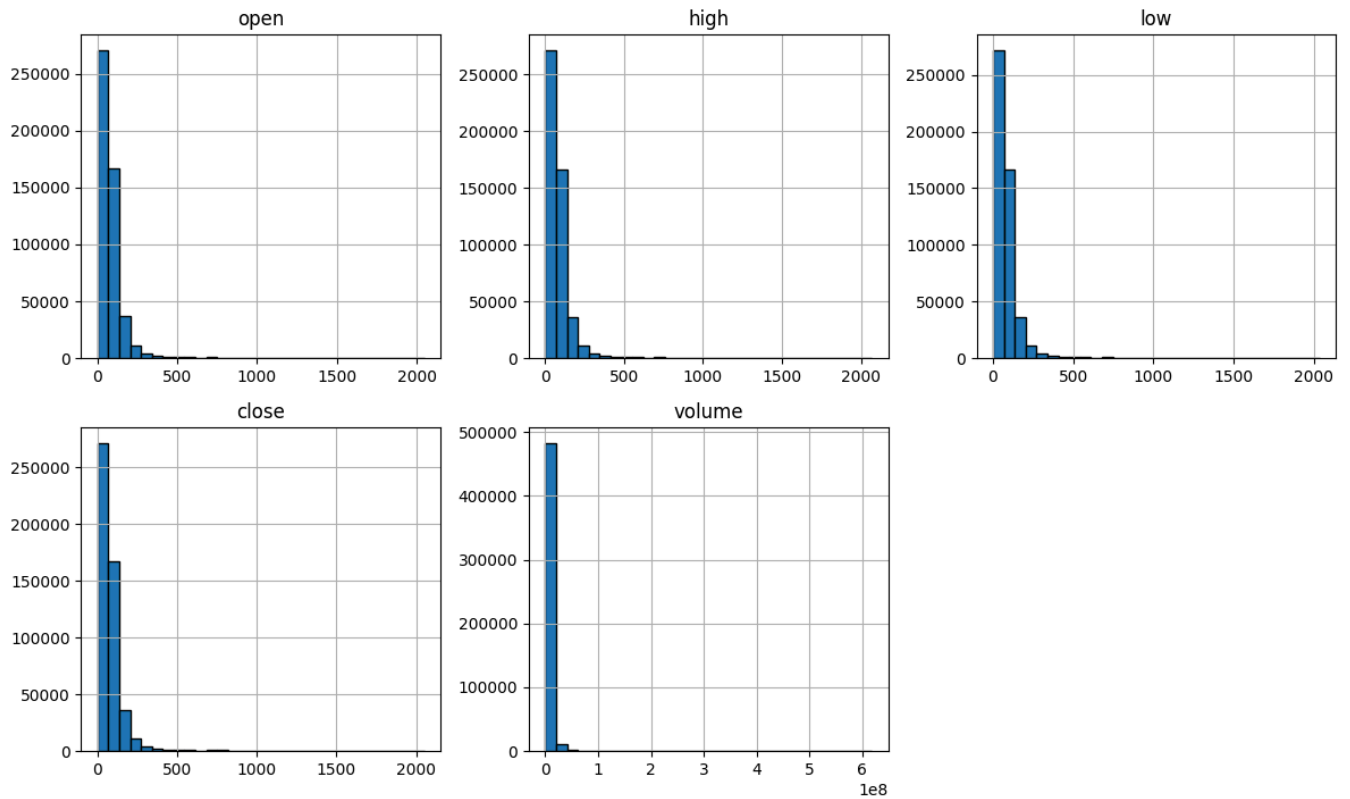
```
df = pd.read_csv("2) Stock Prices Data Set.csv")
```

```
numeric_cols = df.select_dtypes(include='number')
```

```
plt.figure(figsize=(12, 8))
numeric_cols.hist(bins=30, edgecolor='black', layout=(2, 3), figsize=(12, 8))
plt.suptitle("Histograms of All Numeric Features", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

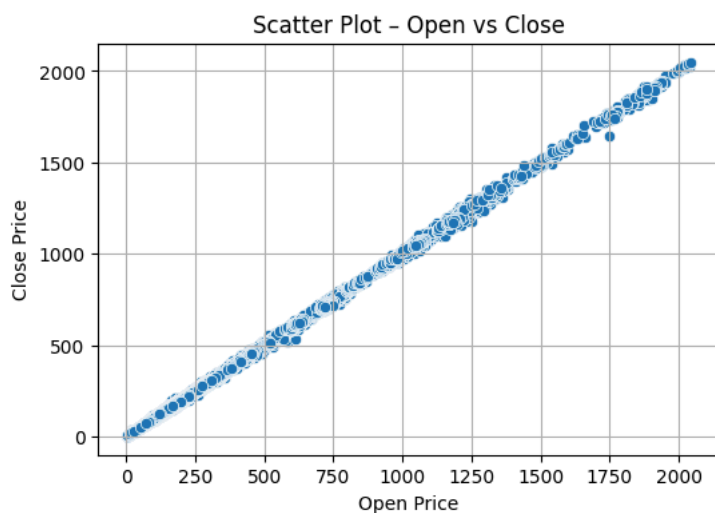
 <Figure size 1200x800 with 0 Axes>

Histograms of All Numeric Features



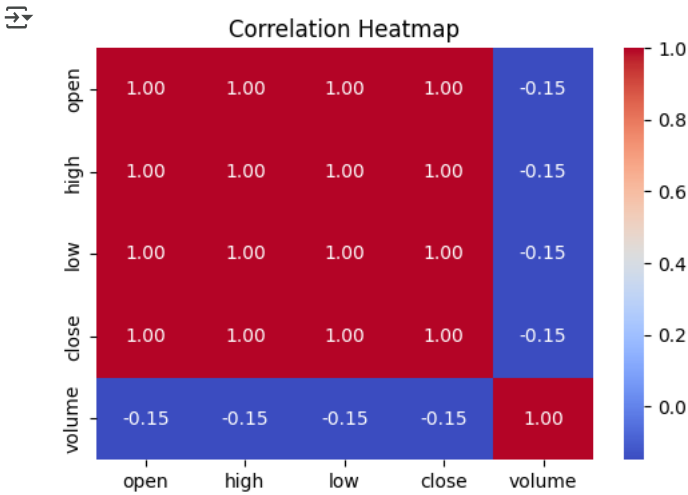
```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='open', y='close')
plt.title("Scatter Plot - Open vs Close")
plt.xlabel("Open Price")
plt.ylabel("Close Price")
plt.grid(True)
plt.show()
```





```
correlation_matrix = numeric_cols.corr()
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



```
from google.colab import files
uploaded = files.upload()
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
df= pd.read_csv("3) Sentiment dataset.csv")
df
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweet
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5
2	2	2	Just finished an amazing workout! 🏋️ ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20
3	3	3	Excited about the upcoming weekend getaway! ...	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Travel #Adventure	8
4	4	4	Trying out a new recipe for dinner tonight. ...	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cooking #Food	12
...	...	...	...	...	...	...	...	...	...
727	728	732	Collaborating on a science project that receiv...	Happy	2017-08-18 18:20:00	ScienceProjectSuccessHighSchool	Facebook	#ScienceFairWinner #HighSchoolScience	20
728	729	733	Attending a surprise birthday party organized ...	Happy	2018-06-22 14:15:00	BirthdayPartyJoyHighSchool	Instagram	#SurpriseCelebration #HighSchoolFriendship	25
729	730	734	Successfully fundraising for a school charity ...	Happy	2019-04-05 17:30:00	CharityFundraisingTriumphHighSchool	Twitter	#CommunityGiving #HighSchoolPhilanthropy	22

```
import pandas as pd

df = pd.read_csv("3) Sentiment dataset.csv")

numeric_cols = df.select_dtypes(include='number')

mean_vals = numeric_cols.mean().round(2)
median_vals = numeric_cols.median().round(2)
```

```

mode_vals = numeric_cols.mode().iloc[0].round(2)
std_vals = numeric_cols.std().round(2)

print("Mean:")
print(mean_vals)

print("Median:")
print(median_vals)

print("Mode:")
print(mode_vals)

print("Standard Deviation:")
print(std_vals)

```

```

↗ Mean:
Unnamed: 0.1      366.46
Unnamed: 0        369.74
Retweets          21.51
Likes             42.90
Year             2020.47
Month             6.12
Day              15.50
Hour             15.52
dtype: float64
Median:
Unnamed: 0.1      366.5
Unnamed: 0        370.5
Retweets          22.0
Likes             43.0
Year             2021.0
Month             6.0
Day              15.0
Hour             16.0
dtype: float64
Mode:
Unnamed: 0.1       0.0
Unnamed: 0         0.0
Retweets          22.0
Likes             45.0
Year             2023.0
Month             2.0
Day              15.0
Hour             14.0
Name: 0, dtype: float64
Standard Deviation:
Unnamed: 0.1      211.51
Unnamed: 0        212.43
Retweets           7.06
Likes             14.09
Year              2.80
Month             3.41
Day              8.47
Hour              4.11
dtype: float64

```

```
import pandas as pd
```

```
df = pd.read_csv("3) Sentiment dataset.csv")
```

```
numeric_cols = df.select_dtypes(include='number')
```

```
correlation_matrix = numeric_cols.corr().round(3)
```

```
print("Correlation Matrix:")
print(correlation_matrix)
```

```

↗ Correlation Matrix:
      Unnamed: 0.1  Unnamed: 0  Retweets  Likes  Year  Month  Day  \
Unnamed: 0.1      1.000      1.000   0.389  0.376  0.102  0.443 -0.080
Unnamed: 0        1.000      1.000   0.389  0.376  0.101  0.444 -0.080
Retweets          0.389   0.389   1.000  0.998 -0.040  0.073  0.009
Likes             0.376   0.376   0.998  1.000 -0.043  0.067  0.011
Year              0.102   0.101  -0.040 -0.043  1.000 -0.315  0.022
Month             0.443   0.444  0.073  0.067 -0.315  1.000 -0.136
Day              -0.080  -0.080  0.009  0.011  0.022 -0.136  1.000
Hour              0.322   0.322  0.197  0.195 -0.087  0.138  0.044

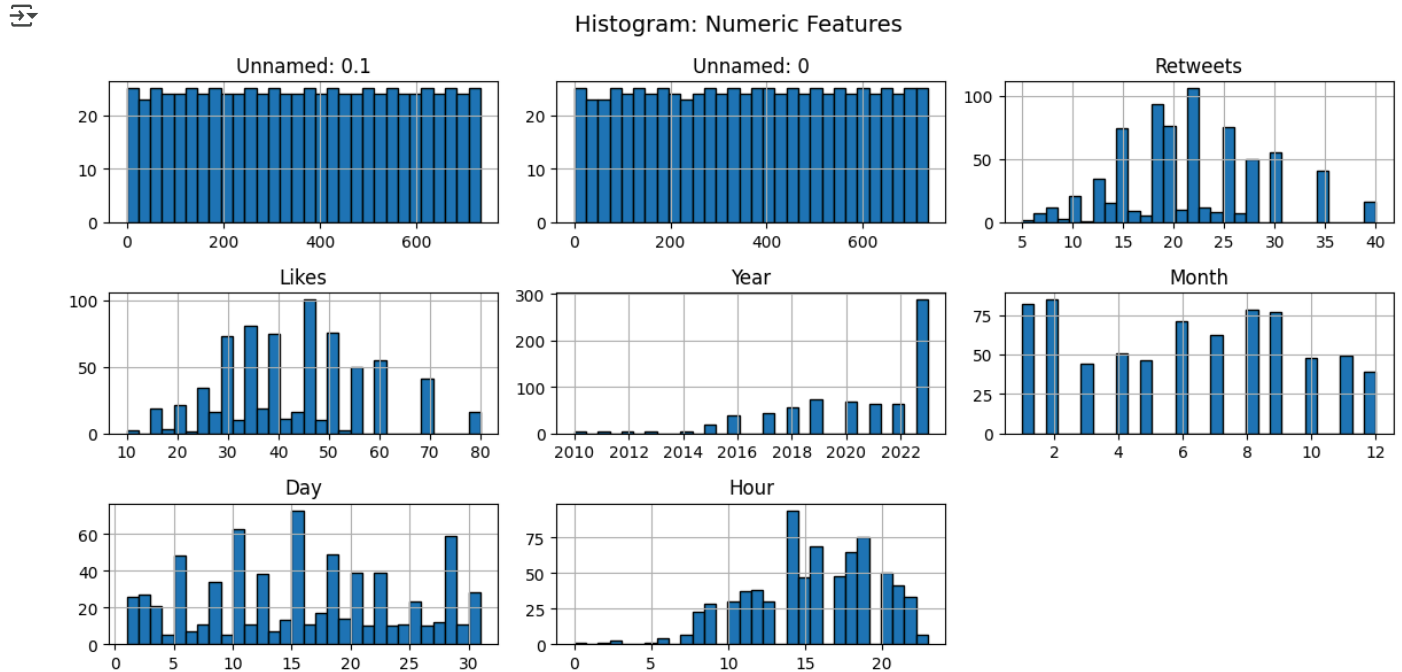
      Hour
Unnamed: 0.1  0.322
Unnamed: 0    0.322
Retweets     0.197
Likes        0.195
Year        -0.087
Month        0.138
Day          0.044
Hour         1.000

```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("3) Sentiment dataset.csv")
```

```
df.select_dtypes(include='number').hist(bins=30, edgecolor='black', figsize=(12, 6))
plt.suptitle("Histogram: Numeric Features", fontsize=14)
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='Likes', y='Retweets', hue='Sentiment', palette='Set2')
plt.title("Scatter Plot: Likes vs Retweets by Sentiment")
plt.xlabel("Likes")
plt.ylabel("Retweets")
plt.grid(True)
plt.show()
```



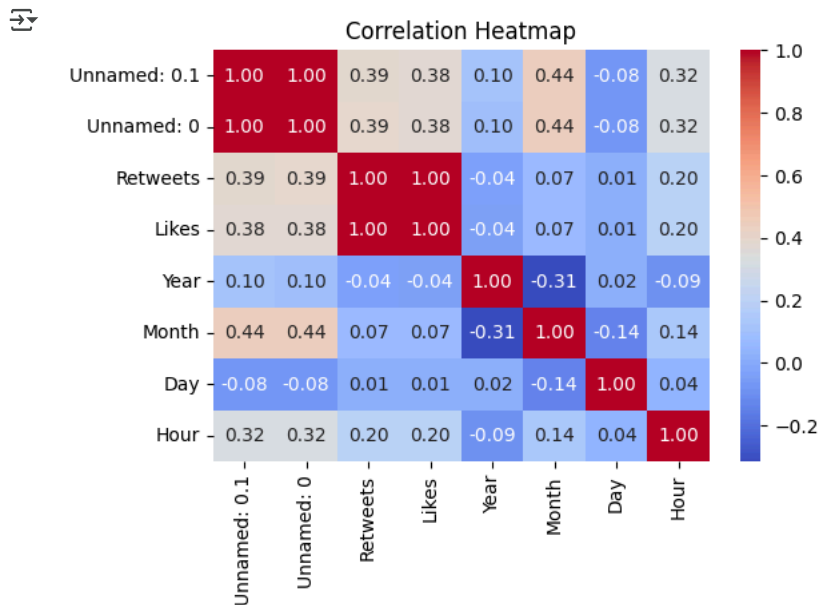
● Nostalgia
● Ambivalence
● Acceptance
● Determination
● Serenity
● Numbness
● Zest
● Contentment
● Hopeful
● Proud
● Grateful
● Empathetic
● Compassionate
● Playful
● Free-spirited
● Inspired
● Confident
● Serenity
● Curiosity
● Ambivalence
● Despair
● Bitterness
● Yearning
● Fearful
● Apprehensive
● Overwhelmed
● Jealous
● Devastated
● Frustrated
● Envious
● Dismissive
● Awe
● Determination
● Nostalgia
● Thrill
● Calmness
● Overwhelmed
● Gratitude
● Bittersweet
● Curiosity
● Admiration
● Overjoyed
● Inspiration
● Motivation
● Amusement
● Contemplation
● JoyfulReunion
● Excitement
● Satisfaction
● Blessed
● Anticipation
● Reflection
● Nostalgia
● Appreciation
● Confidence
● Surprise
● Accomplishment
● Wonderment
● Optimism
● Pride
● Happiness
● Curiosity
● Enchantment
● Intrigue
● PlayfulJoy
● Mindfulness
● DreamChaser
● Elegance
● Whimsy
● Pensive
● Thrill
● Harmony
● Creativity
● Radiance
● Wonder

Rejuvenation
Inspiration
Coziness
Gratitude
Adventure
Euphoria
Awe
Melodic
FestiveJoy
InnerJourney
Freedom
Dazzle
Adrenaline
Harmony
ArtisticBurst
Radiance
Wonder
Inspiration
CulinaryOdyssey
Euphoria
Curiosity
Resilience
Immersion
Nostalgia
Spark
Gratitude
Marvel
Serenity
Heartbreak
Loneliness
Grief
Despair
Betrayal
Suffering
EmotionalStorm
Regret
Isolation
Disappointment
LostLove
Melancholy
Exhaustion
Sorrow
Darkness
Desperation
Ruins
Desolation
Regret
Grief
Heartbreak
Betrayal
Resilience
Sorrow
Loss
Heartache
Solitude
Joy
Happiness
Enthusiasm
Gratitude
Positivity
Kindness
Friendship
Love
Surprise
Success
Thrill
Reflection
Enchantment
Exploration
Awe
Amazement
Romance
Captivation
Wonder
Tranquility

Grandeur
Emotion
Energy
Celebration
Charm
Ecstasy
Hope
Creativity
Colorful
Pride
Hypnotic
Connection
Iconic
Euphoria
Journey
Engagement
Touched
Suspense
Satisfaction
Admiration
Triumph
Heartwarming
Obstacle
Sympathy
Pressure
Renewed Effort
Miscalculation
Challenge
Solace
Breakthrough
Harmony
Joy in Baking
Envisioning History
Imagination
Vibrancy
Mesmerizing
Culinary Adventure
Winter Magic
Thrilling Journey
Nature's Beauty
Celestial Wonder
Creative Inspiration
Runway Creativity
Ocean's Freedom
Whispers of the Past
Boredom
Indifference
Disgust
Relief
Positive
Embarrassed
Mischievous
Sad
Hate
Bad
Neutral
Happy

```
correlation_matrix = df.select_dtypes(include='number').corr()
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



```
import pandas as pd
from scipy import stats

df = pd.read_csv("3) Sentiment dataset.csv")

df_clean = df[['Likes', 'Retweets']].dropna()

slope, intercept, r_value, p_value, std_err = stats.linregress(df_clean['Likes'], df_clean['Retweets'])

print("Linear Regression - Likes vs Retweets\n")
print(f"Regression Equation: y = {slope:.2f} * Likes + ({intercept:.2f})")
print(f"Correlation Coefficient (r): {r_value:.3f}")
print(f"P-value: {p_value:.1e} (Highly significant)" if p_value < 0.05 else f"P-value: {p_value:.1e}")
```

Linear Regression - Likes vs Retweets

Regression Equation: $y = 0.50 * \text{Likes} + (0.04)$
 Correlation Coefficient (r): 0.998
 P-value: 0.0e+00 (Highly significant)

```
from google.colab import files
uploaded = files.upload()
```

No file chosen
 enable.

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE
from sklearn.cluster import KMeans
```

```
from google.colab import files
uploaded = files.upload()
```

 Choose Files churn-bigml-20.csv

- churn-bigml-20.csv(text/csv) - 56329 bytes, last modified: 7/16/2025 - 100% done

Saving churn-bigml-20.csv to churn-bigml-20 (1).csv

```
import pandas as pd
df = pd.read_csv('churn-bigml-20.csv')
df
```



	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	Total night calls	Total night charge	mi
0	LA	117	408	No	No	0	184.5	97	31.37	351.6	80	29.89	215.8	90	9.71	
1	IN	65	415	No	No	0	129.1	137	21.95	228.5	83	19.42	208.8	111	9.40	
2	NY	161	415	No	No	0	332.9	67	56.59	317.8	97	27.01	160.6	128	7.23	
3	SC	111	415	No	No	0	110.4	103	18.77	137.3	102	11.67	189.6	105	8.53	
4	HI	49	510	No	No	0	119.3	117	20.28	215.1	109	18.28	178.7	90	8.04	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
662	WI	114	415	No	Yes	26	137.1	88	23.31	155.7	125	13.23	247.6	94	11.14	
663	AL	106	408	No	Yes	29	83.6	131	14.21	203.9	131	17.33	229.5	73	10.33	
664	VT	60	415	No	No	0	193.9	118	32.96	85.0	110	7.23	210.1	134	9.45	
665	WV	159	415	No	No	0	169.8	114	28.87	197.7	105	16.80	193.7	82	8.72	
666	CT	184	510	Yes	No	0	213.8	105	36.35	159.6	84	13.57	139.2	137	6.26	

667 rows x 20 columns

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
# Data Exploration
print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nData types and missing values:")
print(df.info())
print("\nSummary statistics:")
print(df.describe())
print("\nChurn distribution:")
print(df['Churn'].value_counts(normalize=True))
```



mean	100.537031	30.701709	203.355322	100.470702
std	20.396790	9.436463	49.719268	18.948262
min	30.000000	4.400000	48.100000	37.000000
25%	87.500000	24.860000	171.050000	88.000000
50%	101.000000	30.310000	203.700000	101.000000
75%	115.000000	37.520000	236.450000	113.000000
max	165.000000	56.830000	361.800000	168.000000

	Total eve charge	Total night minutes	Total night calls \
count	667.000000	667.000000	667.000000
mean	17.285262	199.685307	100.113943
std	4.226160	49.759931	20.172505
min	4.090000	23.200000	42.000000
25%	14.540000	167.950000	86.000000
50%	17.310000	201.600000	100.000000
75%	20.095000	231.500000	113.500000
max	30.750000	367.700000	175.000000

	Total night charge	Total intl minutes	Total intl calls \
count	667.000000	667.000000	667.000000
mean	8.985907	10.238381	4.527736
std	2.239429	2.807850	2.482442
min	1.040000	0.000000	0.000000
25%	7.560000	8.600000	3.000000
50%	9.070000	10.500000	4.000000
75%	10.420000	12.050000	6.000000
max	16.550000	18.300000	18.000000

	Total intl charge	Customer service calls
count	667.000000	667.000000
mean	2.764948	1.563718
std	0.758167	1.333357
min	0.000000	0.000000
25%	2.320000	1.000000
50%	2.840000	1.000000
75%	3.255000	2.000000
max	4.940000	8.000000

Churn distribution:

Churn

False 0.857571

True 0.142429

Name: proportion, dtype: float64

Visualizations

plt.figure(figsize=(15, 10))

↗ <Figure size 1500x1000 with 0 Axes>
 ↗ <Figure size 1500x1000 with 0 Axes>

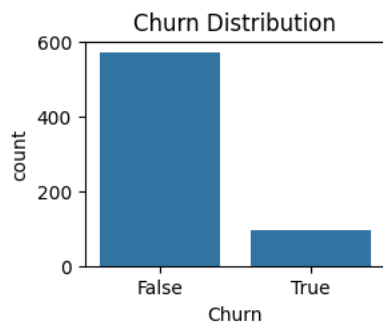
Churn distribution

plt.subplot(2, 2, 1)

sns.countplot(x='Churn', data=df)

plt.title('Churn Distribution')

↗ Text(0.5, 1.0, 'Churn Distribution')



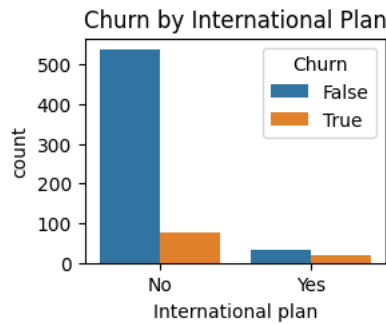
International plan vs churn

plt.subplot(2, 2, 2)

sns.countplot(x='International plan', hue='Churn', data=df)

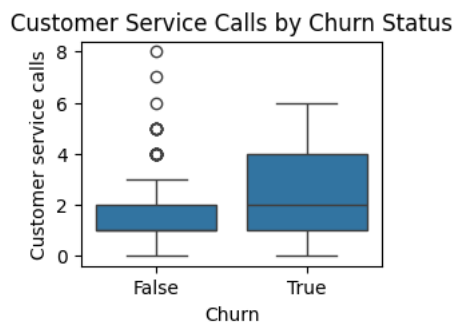
plt.title('Churn by International Plan')

```
Text(0.5, 1.0, 'Churn by International Plan')
```

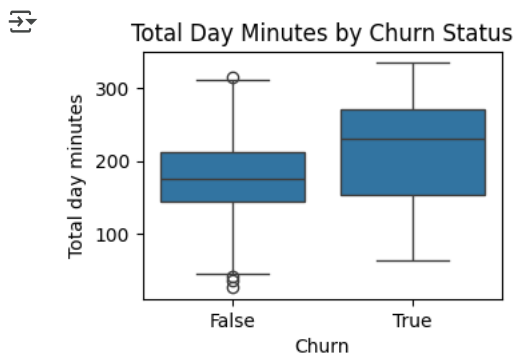


```
# Customer service calls vs churn
plt.subplot(2, 2, 3)
sns.boxplot(x='Churn', y='Customer service calls', data=df)
plt.title('Customer Service Calls by Churn Status')
```

```
Text(0.5, 1.0, 'Customer Service Calls by Churn Status')
```



```
# Total day minutes vs churn
plt.subplot(2, 2, 4)
sns.boxplot(x='Churn', y='Total day minutes', data=df)
plt.title('Total Day Minutes by Churn Status')
plt.tight_layout()
plt.show()
```



```
#Feature Engineering
# Create new features that might be predictive
df['Total minutes'] = df['Total day minutes'] + df['Total eve minutes'] + df['Total night minutes']
df['Total calls'] = df['Total day calls'] + df['Total eve calls'] + df['Total night calls']
df['Total charge'] = df['Total day charge'] + df['Total eve charge'] + df['Total night charge']
df['Avg minutes per call'] = df['Total minutes'] / df['Total calls']
df['Charge per minute'] = df['Total charge'] / df['Total minutes']

# Data Preprocessing
# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Define categorical and numerical features
categorical_features = ['State', 'International plan', 'Voice mail plan']
numerical_features = [col for col in X.columns if col not in categorical_features + ['Area code']]
```

```

# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

# Handle class imbalance with SMOTE
smote = SMOTE(random_state=42)

# Model Building
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier()
}

# Evaluate models
results = {}
for name, model in models.items():
    # Create pipeline
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', model) # SMOTE will be applied outside the pipeline
    ])

    # Fit the pipeline on the training data
    X_train_processed = preprocessor.fit_transform(X_train)

    # Apply SMOTE to the preprocessed training data
    X_train_smote, y_train_smote = smote.fit_resample(X_train_processed, y_train)

    # Fit the classifier on the SMOTE-augmented training data
    pipeline.named_steps['classifier'].fit(X_train_smote, y_train_smote)

    # Make predictions
    y_pred = pipeline.predict(X_test)
    y_prob = pipeline.predict_proba(X_test)[: , 1]

    # Store results
    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'ROC AUC': roc_auc_score(y_test, y_prob)
    }

# Convert results to DataFrame
results_df = pd.DataFrame(results).T
print("\nModel Performance Comparison:")
print(results_df)

```

↗

Model Performance Comparison:					
	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.828358	0.428571	0.631579	0.510638	0.826087
Random Forest	0.932836	0.857143	0.631579	0.727273	0.920824
Gradient Boosting	0.970149	1.000000	0.789474	0.882353	0.908467
SVM	0.940299	0.823529	0.736842	0.777778	0.914416
KNN	0.798507	0.394737	0.789474	0.526316	0.854691

```

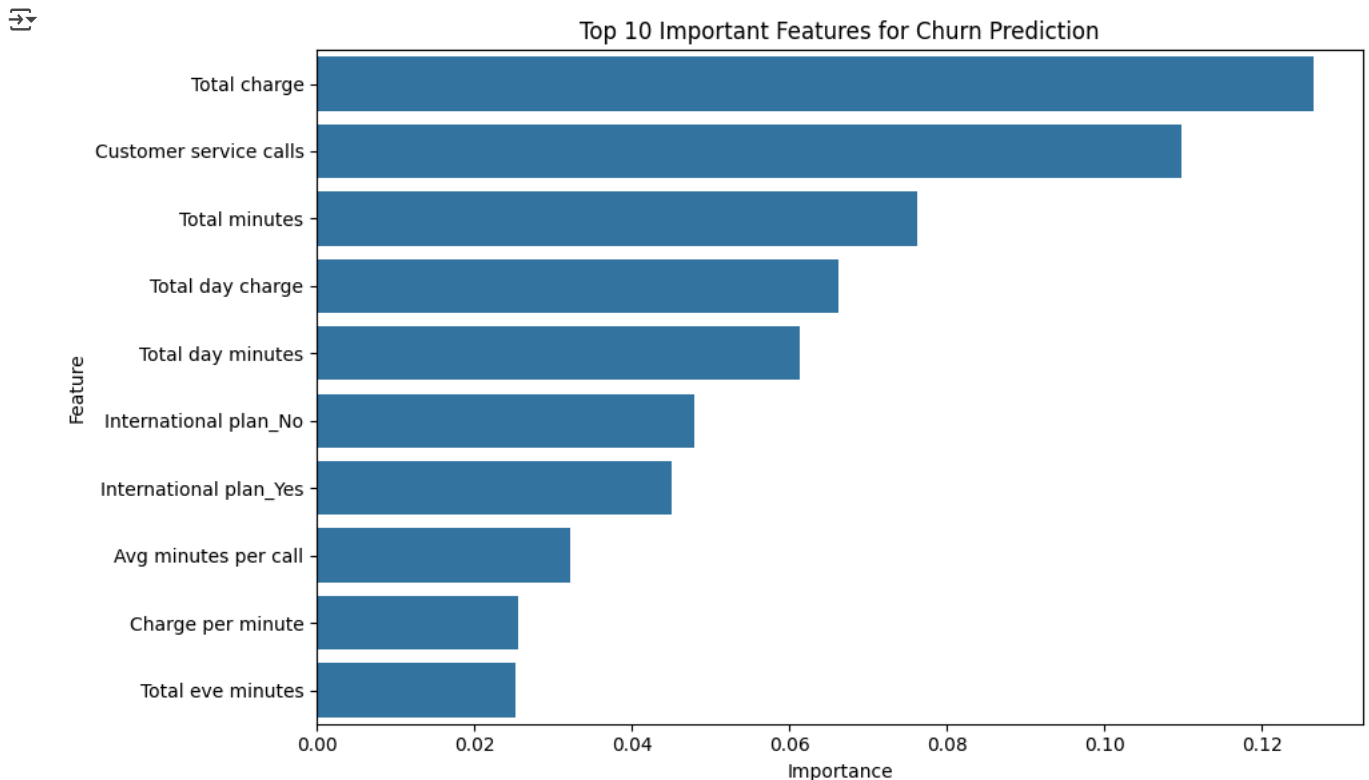
# Feature Importance Analysis
# Get feature names after one-hot encoding
preprocessor.fit(X_train)
cat_features = rf_pipeline.named_steps['preprocessor'].named_transformers_['cat'].get_feature_names_out(categorical_features)
feature_names = numerical_features + list(cat_features)

# Get feature importances from the Random Forest model
importances = rf_pipeline.named_steps['classifier'].feature_importances_

# Create a DataFrame for feature importance
feature_importance = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance = feature_importance.sort_values('Importance', ascending=False)

```

```
# Plot top 10 most important features
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance.head(10))
plt.title('Top 10 Important Features for Churn Prediction')
plt.tight_layout()
plt.show()
```

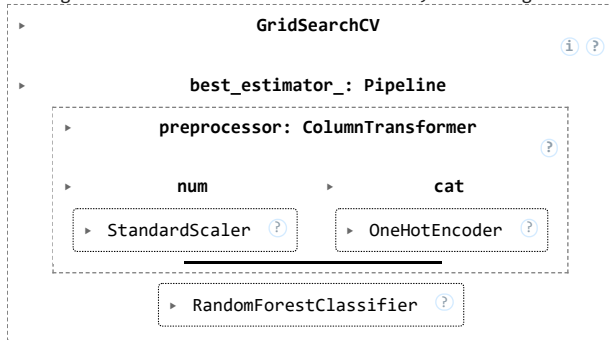


```
# Hyperparameter Tuning for Random Forest
param_grid = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5],
    'classifier__min_samples_leaf': [1, 2]
}
```

```
# Set up GridSearchCV
grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1
)
```

```
# Fit the grid search to the data
grid_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 24 candidates, totalling 120 fits



```
# Print best parameters and score
print("\nBest parameters:", grid_search.best_params_)
print("Best ROC AUC score:", grid_search.best_score_)
```

```
Best parameters: {'classifier__max_depth': None, 'classifier__min_samples_leaf': 1, 'classifier__min_samples_split': 2, 'classifier__min_samples_split': 2, 'classifier__min_samples_split': 2}
Best ROC AUC score: 0.9109332696289218
```

```
# Evaluate the tuned model
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)
y_prob_tuned = best_model.predict_proba(X_test)[:, 1]
```

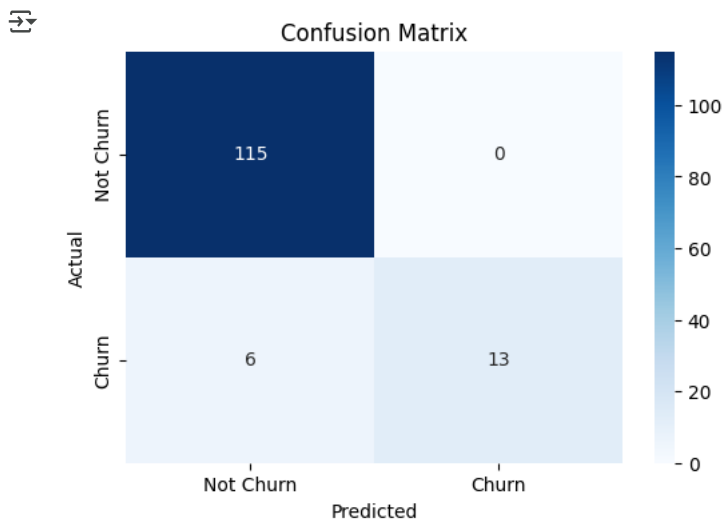
```
# Classification report
print("\nClassification Report for Tuned Model:")
print(classification_report(y_test, y_pred_tuned))
```

```
Classification Report for Tuned Model:
              precision    recall  f1-score   support

   False      0.95      1.00      0.97       115
    True      1.00      0.68      0.81        19

 accuracy      0.96
 macro avg      0.98
 weighted avg    0.96
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_tuned)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Churn', 'Churn'],
            yticklabels=['Not Churn', 'Churn'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```



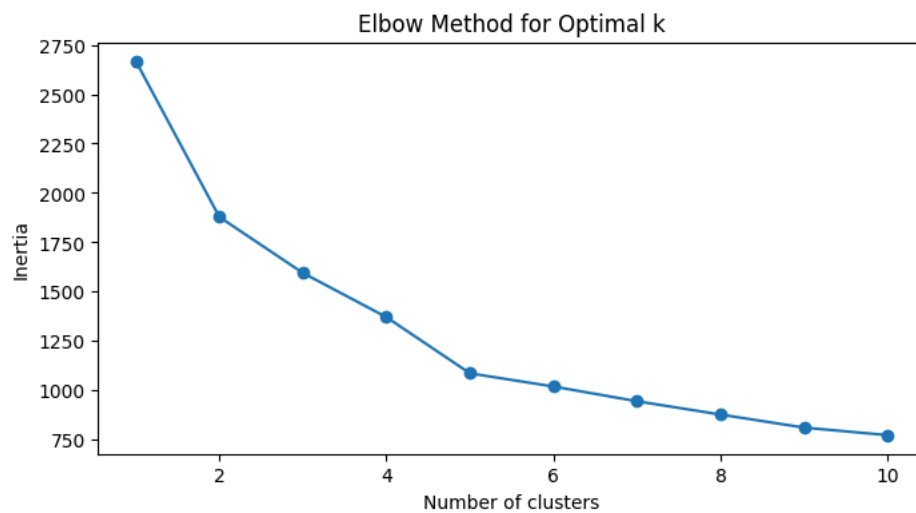
```
# Market Segmentation using Clustering
# Select features for clustering
cluster_features = ['Total minutes', 'Total charge', 'Customer service calls', 'Total intl minutes']
X_cluster = df[cluster_features]
```

```
# Standardize the features
scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)
```

```
# Determine optimal number of clusters using elbow method
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_cluster_scaled)
    inertia.append(kmeans.inertia_)
```

```
# Plot the elbow curve
plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
```

```
plt.ylabel('Inertia')
plt.show()
```



```
# Based on the elbow plot, choose optimal k (e.g., 3)
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(X_cluster_scaled)
```

```
# Add clusters to original dataframe
df['Cluster'] = clusters
```

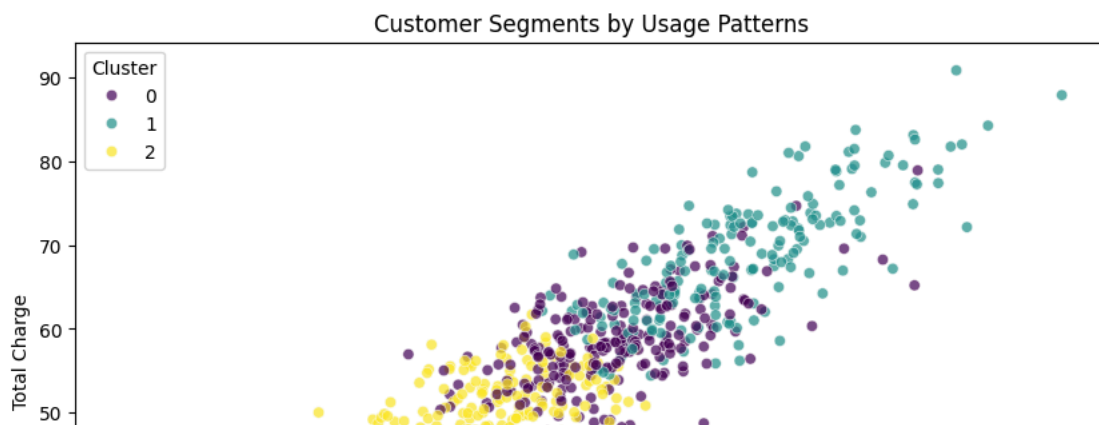
```
# Analyze cluster characteristics
# Select only numeric columns for aggregation
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
cluster_analysis = df.groupby('Cluster')[numeric_cols + ['Churn']].mean()
print("\nCluster Analysis:")
print(cluster_analysis[cluster_features + ['Churn']])
```



```
Cluster Analysis:
      Total minutes  Total charge  Customer service calls \
Cluster
0          599.535931      58.753247                0.965368
1          677.833520      68.422123                1.815642
2          504.651751      47.554125                1.926070

      Total intl minutes      Churn
Cluster
0          12.071429  0.056277
1           8.575419  0.279330
2           9.749027  0.124514
```

```
# Visualize clusters in 2D space
plt.figure(figsize=(10, 6))
sns.scatterplot(
    x='Total minutes',
    y='Total charge',
    hue='Cluster',
    data=df,
    palette='viridis',
    alpha=0.7
)
plt.title('Customer Segments by Usage Patterns')
plt.xlabel('Total Minutes')
plt.ylabel('Total Charge')
plt.show()
```



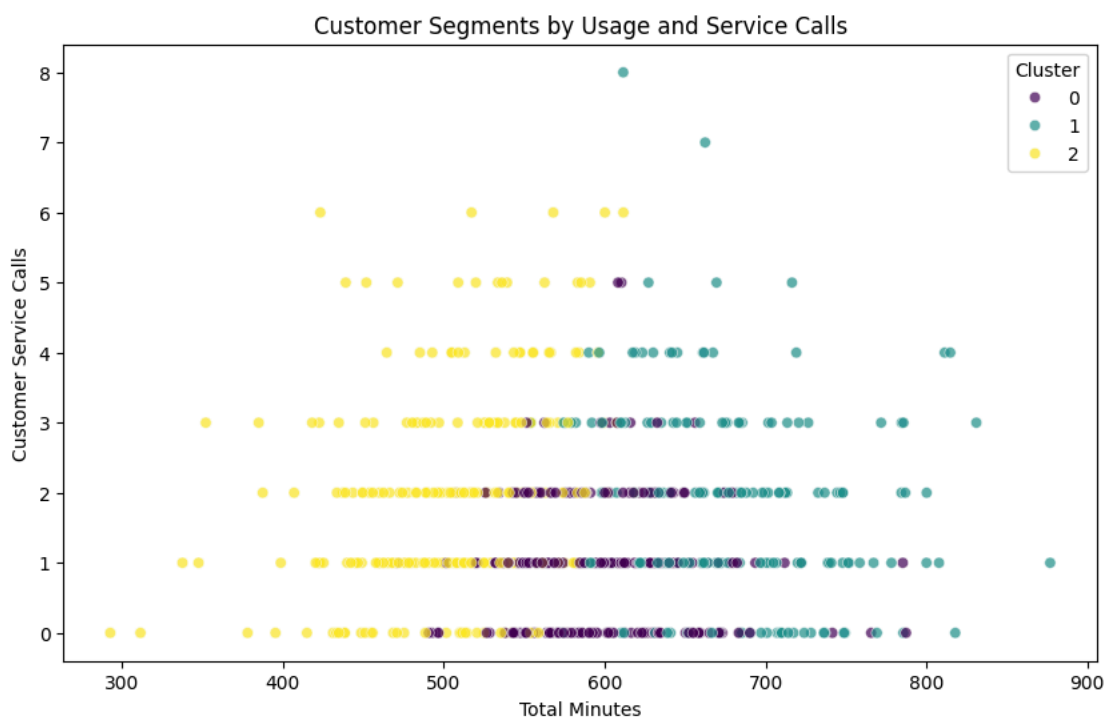
Visualization with customer service calls

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
    x='Total minutes',
    y='Customer service calls',
    hue='Cluster',
    data=df,
    palette='viridis',
    alpha=0.7
)
```

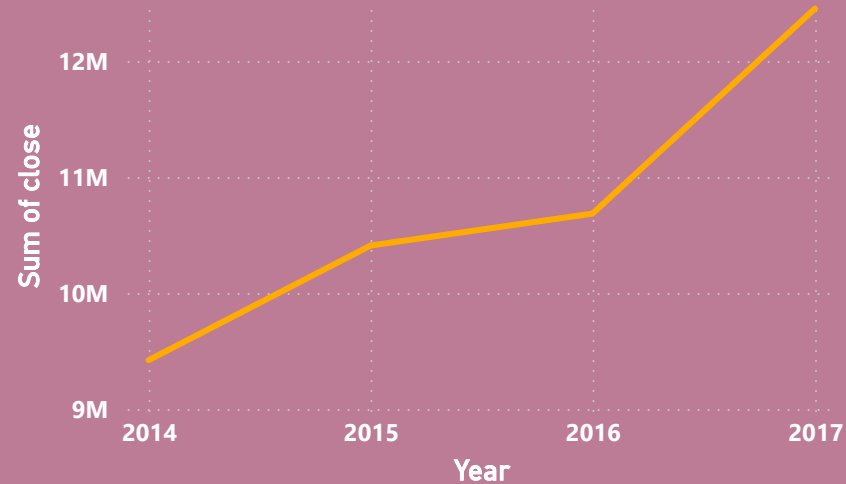
```
plt.title('Customer Segments by Usage and Service Calls')
plt.xlabel('Total Minutes')
```

```
plt.ylabel('Customer Service Calls')
plt.show()
```

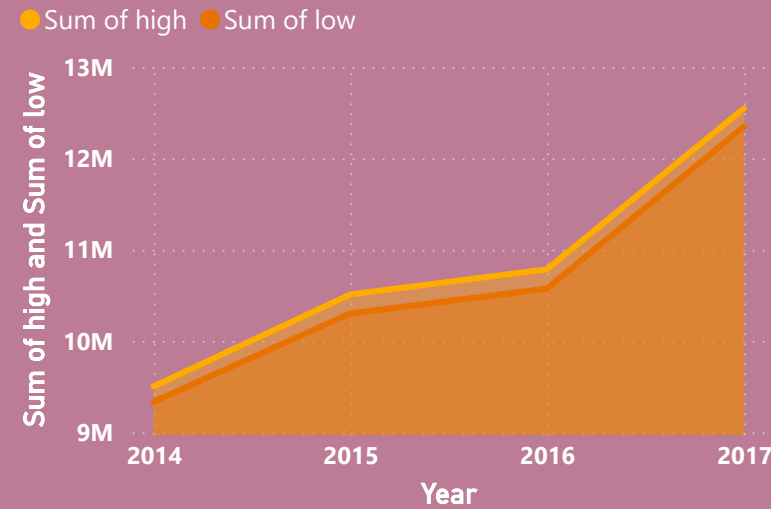


STOCK PRICE DATA SET DASHBOARD

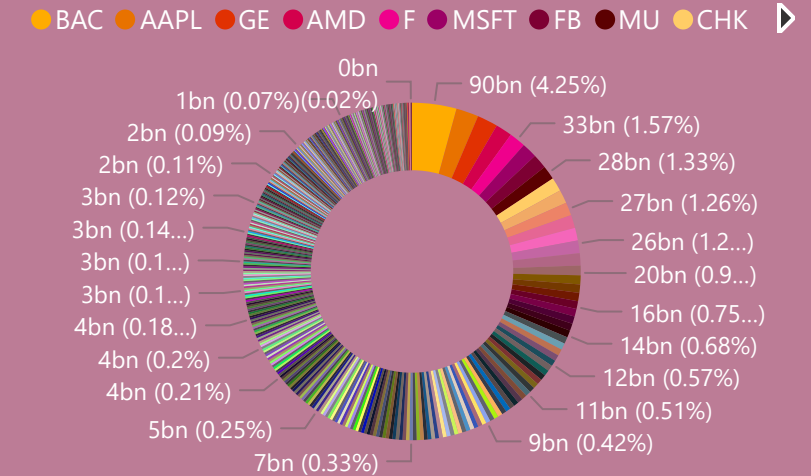
Sum of close by Year



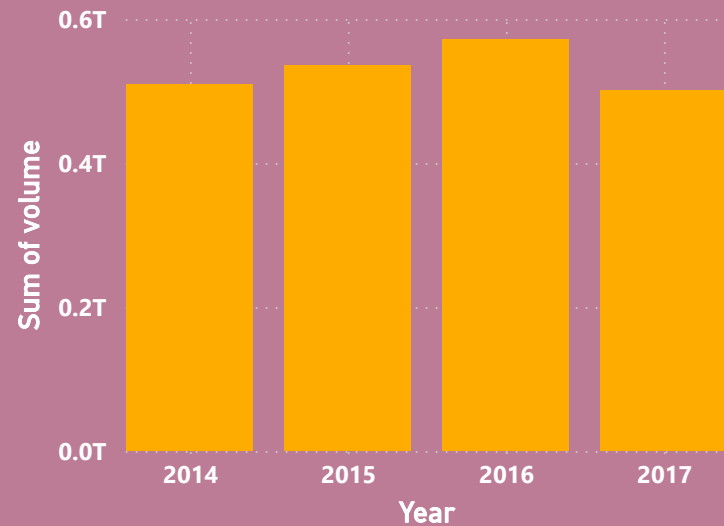
Sum of high and Sum of low by Year



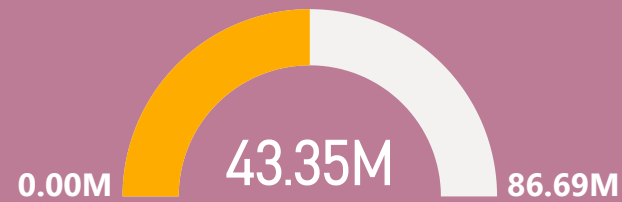
Sum of volume by symbol



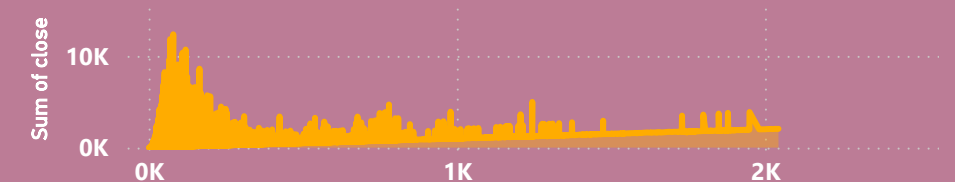
Sum of volume by Year



Sum of high



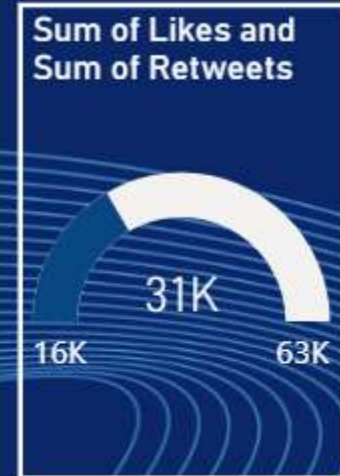
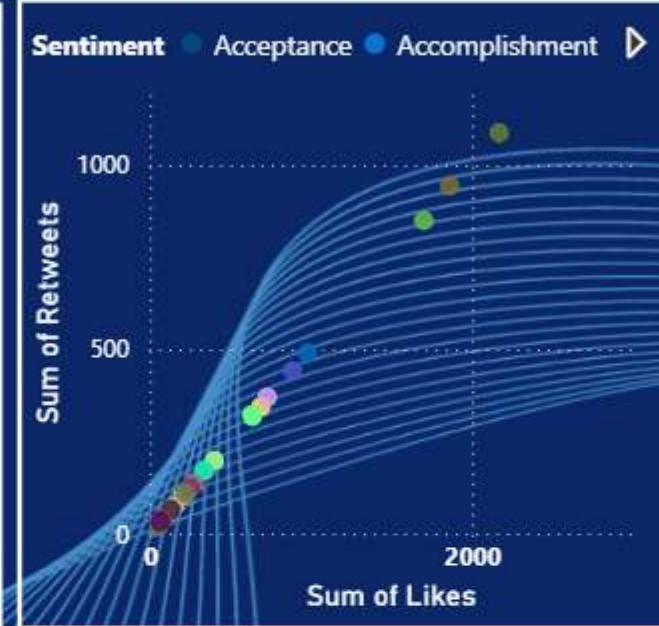
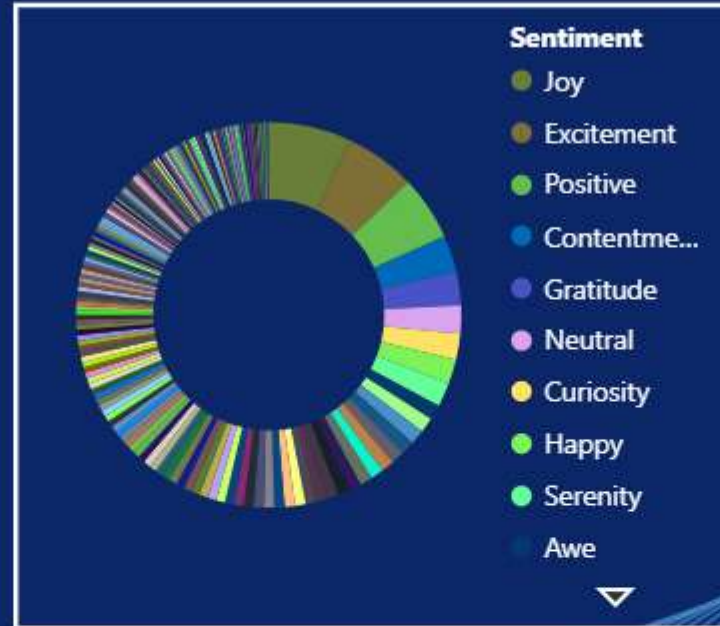
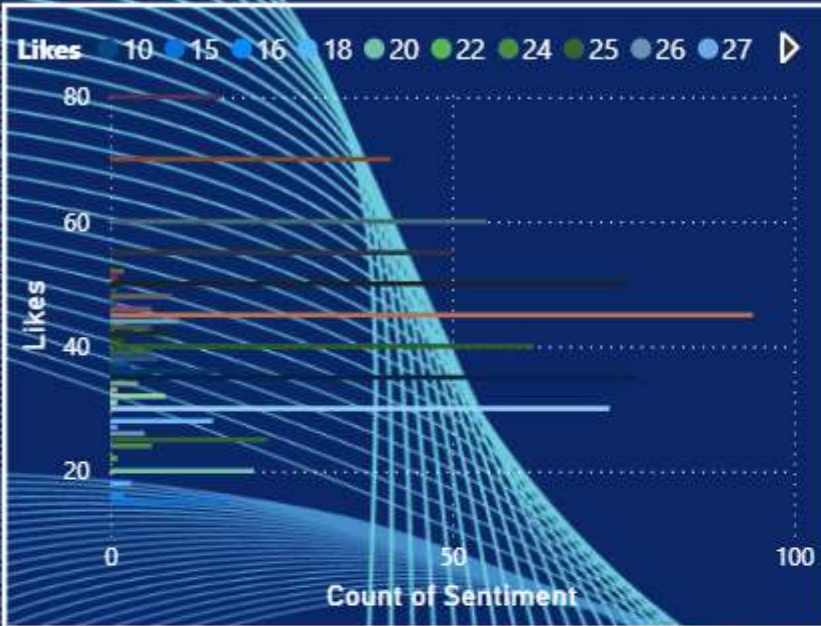
Sum of close by open



Year	Sum of open	Sum of high	Sum of low	Sum of close	Sum of volume
2017	12,448.98K	1,25,47,076.90	1,23,48,644.21	1,24,52,000.89	500400300045
2014	9,420.41K	95,03,775.34	93,34,875.43	94,20,877.06	509651540155
2015	10,408.45K	1,05,09,728.47	1,03,02,518.81	1,04,09,063.14	534954787348
2016	10,679.05K	1,07,84,732.21	1,05,73,234.22	1,06,84,258.91	571045692991
Total	42,956.89K	4,33,45,312.91	4,25,59,272.67	4,29,66,200.00	2116052320539

2T 2.05K
Sum of volume Max of close

SENTIMENT DATASET DASHBOARD



31K

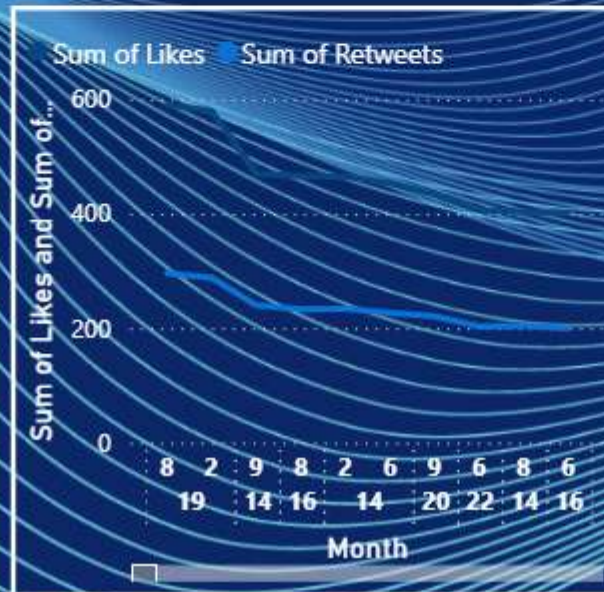
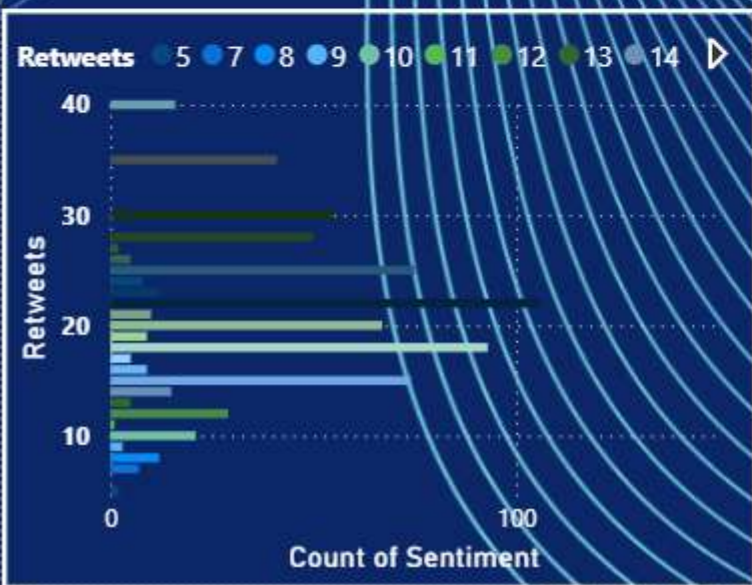
Sum of Likes

16K

Sum of Retweets

Year, Month, Sentiment ▾

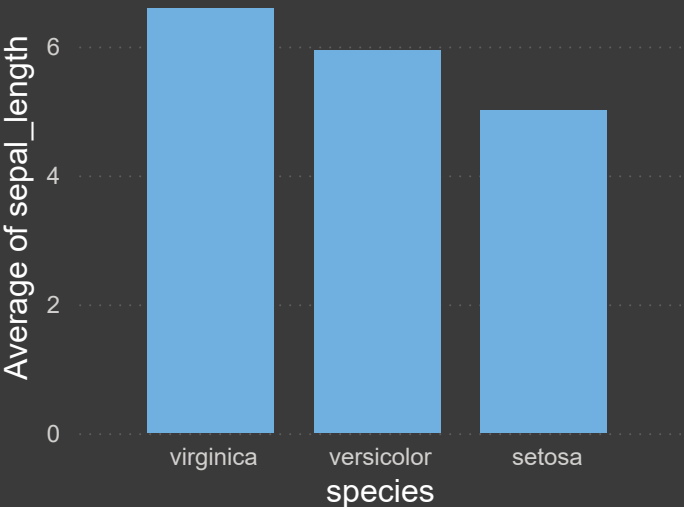
All ▾



Sentiment	Sum of Likes	Sum of Retweets
Acceptance	273	13
Accomplishment	155	13
Admiration	175	8
Adoration	90	4
Adrenaline	45	2
Adventure	185	9
Affection	60	3
Amazement	60	3
Ambivalence	168	8
Amusement	100	7
Total	31404	1574

IRIS DATASET DASHBOARD

Average of sepal_length by species



34.83

5.84

1.20

3.76

3.05

Average of Sepal_Area

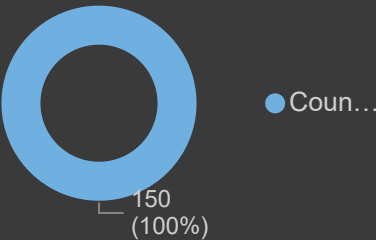
Average of sepal_len...

Average of petal_width

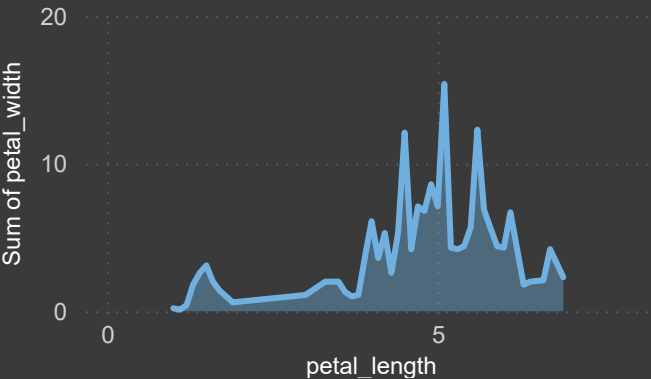
Average of petal_length

Average of sepal_width

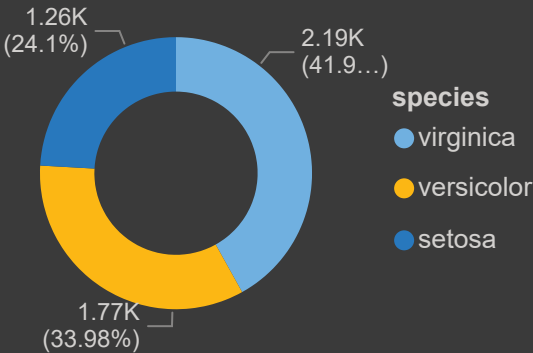
Count of species



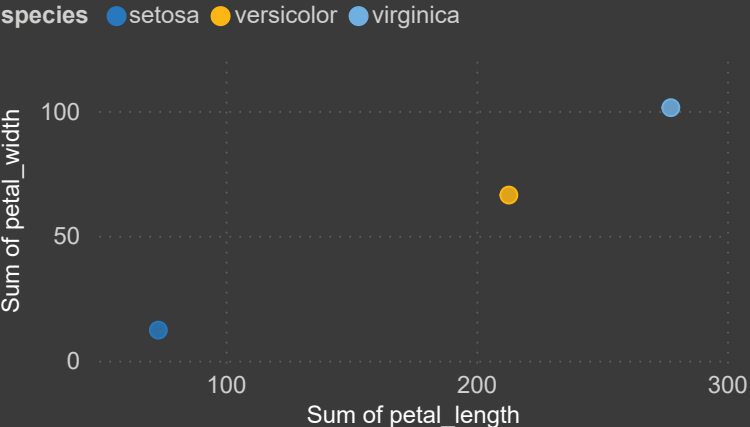
Sum of petal_width by petal_length



Sum of Sepal_Area by species



Sum of petal_length and Sum of petal_width by species



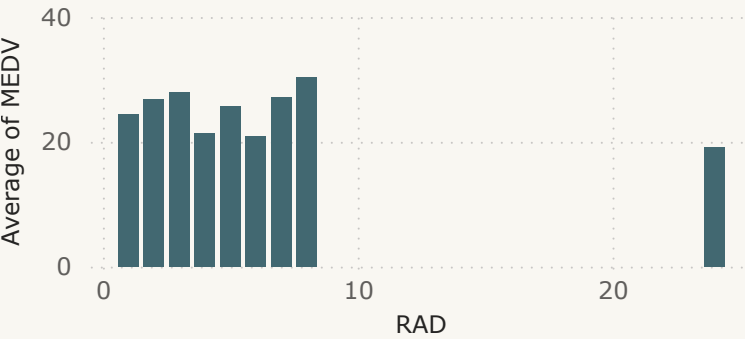
Sum of petal_length and Sum of petal_width



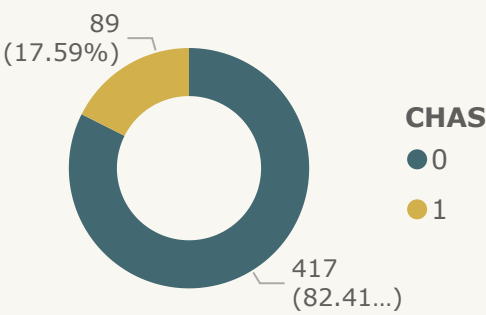
Sum of petal_length	Sum of petal_width	Sum of sepal_length	Sum of sepal_width	species
277.60	101.30	329.40	148.70	virginica
213.00	66.30	296.80	138.50	versicolor
73.20	12.20	250.30	170.90	setosa
563.80	179.80	876.50	458.10	

HOUSE PREDICTION DATA SET DASHBOARD

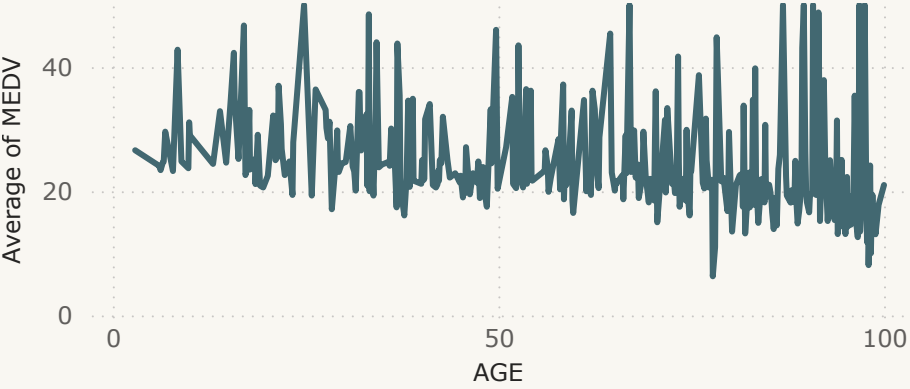
Avg House Value by Highway Access



Houses Near Charles River



Average of MEDV by AGE



11.54 78.06 23.75 1.27

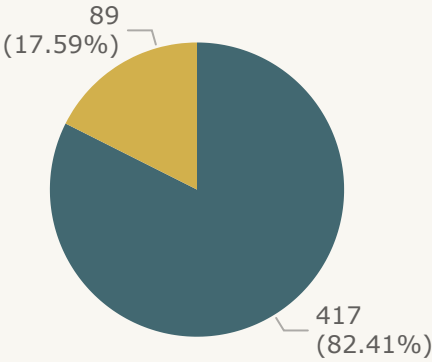
Average of LSTAT

Average of RAD

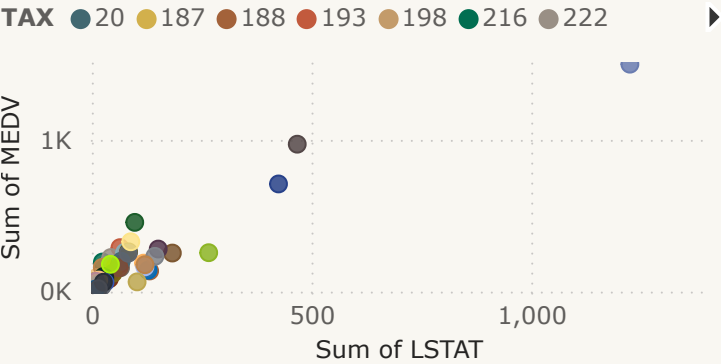
Average of MEDV

Average of CRIM

Houses Near Charles River



House Value vs Lower Income Population



MEDV	RM	LSTAT	CRIM	TA X	DIS	RAD	CR
7.00	5.41	23.97	0.18	711	1.76	4	
8.10	5.09	29.68	0.21	711	1.82	4	
13.60	5.98	18.07	0.11	711	1.87	4	
15.20	5.45	18.06	0.15	711	1.82	4	
20.10	5.98	13.35	0.11	711	2.11	4	
6.30	5.85	29.97	9.92	666	1.50	24	
8.50	5.75	19.92	7.67	666	1.63	24	
9.50	6.38	24.08	9.34	666	1.97	24	
11.00	6.19	21.52	7.37	666	1.94	24	
11.30	5.54	23.60	9.19	666	1.58	24	
11.50	5.39	20.85	8.15	666	1.73	24	
11.70	5.57	17.16	8.79	666	2.06	24	
12.10	6.40	20.31	9.60	666	1.64	24	
12.30	5.52	24.56	7.99	666	1.53	24	
12.50	6.41	19.37	5.87	666	1.68	24	

MY SQL PROJECT QUERIES USING IRIS DATASET

```
CREATE DATABASE DATA;
```

```
USE DATA;
```

```
CREATE TABLE IRIS_DATA(
```

```
    SEPAL_LENGTH FLOAT,
```

```
    SEPAL_WIDTH FLOAT,
```

```
    PETAL_LENGTH FLOAT,
```

```
    PETAL_WIDTH FLOAT,
```

```
    SPECIES VARCHAR(50)
```

```
);
```

```
SHOW DATABASES;
```

```
SHOW TABLES;
```

```
## SELECT QUERY ##
```

```
SELECT * FROM IRIS_DATA;
```

```
## FILTERING ROWS THROUGH WHERE CLAUSE ##
```

```
SELECT * FROM IRIS_DATA WHERE SPECIES = 'SETOSA';
```

```
## AVERAGE ##
```

```
SELECT AVG(SEPAL_LENGTH) AS AVG_SEPAL FROM IRIS_DATA;
```

```
## MAXIMUM
```

```
SELECT MAX(PETAL_WIDTH) AS MAX_PETAL FROM IRIS_DATA;
```

```
## MINIMUM ##
```

```
SELECT MIN(SEPAL_WIDTH) AS MIN_SEPAL FROM IRIS_DATA;
```

```
## COUNT ##
```

```
SELECT COUNT(*) AS TOTAL_ROWS FROM IRIS_DATA;
```

```
## GROUPBY ##
```

```
SELECT SPECIES, AVG(SEPAL_LENGTH) AS AVG_SEPAL_LENGTH FROM  
IRIS_DATA GROUP BY SPECIES;
```

```
SELECT SPECIES, AVG(PETAL_LENGTH) AS AVG_PETAL_LENGTH FROM  
IRIS_DATA GROUP BY SPECIES;
```

```
## SUM ##
```

```
SELECT SUM(SEPAL_LENGTH) AS TOTAL_SEPAL_LENGTH FROM IRIS_DATA;
```

```
SELECT SUM(PETAL_WIDTH) AS SUM_PETAL_WIDTH FROM IRIS_DATA WHERE  
SEPAL_LENGTH > 6;
```

```
## JOIN TABLE ##
```

```
CREATE TABLE SPECIES_INFO(  
SPECIES VARCHAR(50),  
FLOWER_COLOR VARCHAR(20));
```

```
INSERT INTO SPECIES_INFO(SPECIES, FLOWER_COLOR) VALUES  
( 'SETOSA','BLUE'),  
( 'VERSICOLOR','PURPLE'),  
( 'VIRGINICA','WHITE');
```

```
SELECT * FROM SPECIES_INFO;
```

```
SELECT I.SEPAL_LENGTH,I.PETAL_LENGTH,I.SPECIES,S.FLOWER_COLOR FROM  
IRIS_DATA I JOIN
```

```
SPECIES_INFO S ON I.SPECIES= S.SPECIES;
```

```
## ORDERBY - SORTING ##
```

```
SELECT * FROM IRIS_DATA ORDER BY PETAL_LENGTH DESC LIMIT 10;
```

```
## SUBQUERY - FIND SPECIES WITH HIGHEST AVERAGE SEPAL LENGTH ##
```

```
SELECT SPECIES FROM IRIS_DATA GROUP BY SPECIES ORDER BY  
AVG(SEPAL_LENGTH) DESC LIMIT 1;
```

```
## CREATE INDEXES (IF FILTERING/SEARCHING A COLUMN OFTEN) ##
```

```
CREATE INDEX IDX_SPECIES ON IRIS_DATA(SPECIES);
```

```
## USE LIMIT TO REDUCE THE OUTPUT SIZE ##
```

```
SELECT * FROM IRIS_DATA LIMIT 10;
```

```
## USE WHERE CLAUSE TO FILTER BEFORE GROUP BY OR JOIN ##
```

```
SELECT SPECIES, AVG(SEPAL_LENGTH) FROM IRIS_DATA WHERE  
SEPAL_LENGTH > 5 GROUP BY SPECIES;
```

MY SQL PROJECT QUERIES USING HOUSE PREDICTION

DATASET

```
CREATE DATABASE HOUSE;
```

```
USE HOUSE;
```

```
CREATE TABLE house_prediction_data (
```

```
    CRIM FLOAT,
```

```
    ZN FLOAT,
```

```
    INDUS FLOAT,
```

```
    CHAS INT,
```

```
    NOX FLOAT,
```

```
    RM FLOAT,
```

```
    AGE FLOAT,
```

```
    DIS FLOAT,
```

```
    RAD INT,
```

```
    TAX FLOAT,
```

```
    PTRATIO FLOAT,
```

```
    B FLOAT,
```

```
    LSTAT FLOAT,
```

```
    MEDV FLOAT
```

```
);
```

```
select * from house_prediction_data;
```

```
## TOTAL OF HOUSE PRICES
```

```
SELECT SUM(MEDV) AS total_house_value FROM house_prediction_data;
```

```
## TOTAL TAX COLLECTED
```

```
SELECT SUM(TAX) AS total_tax FROM house_prediction_data;
```

```
## TOTAL OF ALL NUMERIC COLUMNS ##
```

```
SELECT SUM(CRIM) AS total_crime_rate, SUM(RM) AS total_rooms, SUM(TAX) AS  
total_tax,
```

```
SUM(MEDV) AS total_value FROM house_prediction_data;
```

```
## Average House Price ##
```

```
SELECT AVG(MEDV) AS avg_price FROM house_prediction_data;
```

```
## AVERAGE PRICE BY ACCESS TO CHARLES RIVER ##
```

```
SELECT CHAS, AVG(MEDV) AS avg_price FROM house_prediction_data GROUP BY  
CHAS;
```

```
## TOP 10 HIGH-VALUE HOUSES ##
```

```
SELECT * FROM house_prediction_data ORDER BY MEDV DESC LIMIT 10;
```

```
## TOTAL COUNT BY NUMBER OF ROOMS ##
```

```
SELECT RM, COUNT(*) AS house_count FROM house_prediction_data GROUP BY RM  
ORDER BY RM;
```

```
## JOIN EXAMPLE ##
```

```
CREATE TABLE crime_info (  
    CRIM FLOAT,  
    risk_level VARCHAR(20)  
);
```

```
INSERT INTO crime_info (CRIM, risk_level)
```

```
VALUES (0.00632, 'Low'), (0.02731, 'Medium'), (0.03237, 'High');
```

```
SELECT h.CRIM, h.MEDV, c.risk_level FROM house_prediction_data h JOIN  
crime_info c ON h.CRIM = c.CRIM;
```

```
## QUERY OPTIMIZATION ##
```

```
CREATE INDEX idx_medv ON house_prediction_data(MEDV);
```

```
CREATE INDEX idx_chas ON house_prediction_data(CHAS);
```

```
SELECT AVG(MEDV) FROM house_prediction_data WHERE RM > 6;
```